Natural Computing Lab,
National University of Science and Technology
Politehnica Bucuresti

I-RDT: A Tiny-ML Oriented Feature Extractor with Applications in Signal Classification on Low Complexity Computational Platforms

Radu Dogaru, Ioana Dogaru –
Dept. of Applied Electronics and Information Engineering

Outline

Introduction

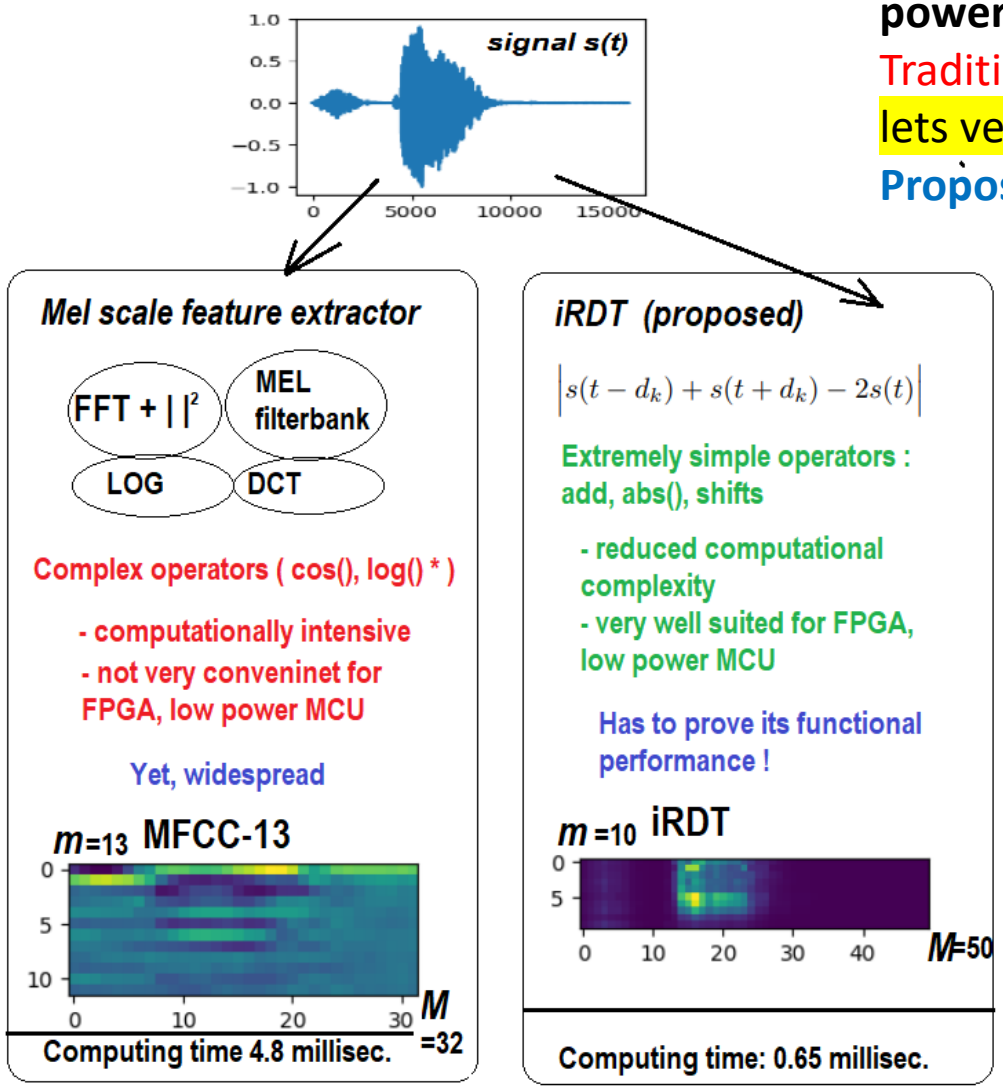
Challenges and applicative areas

iRDT algorithm principles and TinyML classifiers

Optimization of models, results in 4 study cases

Conclusions

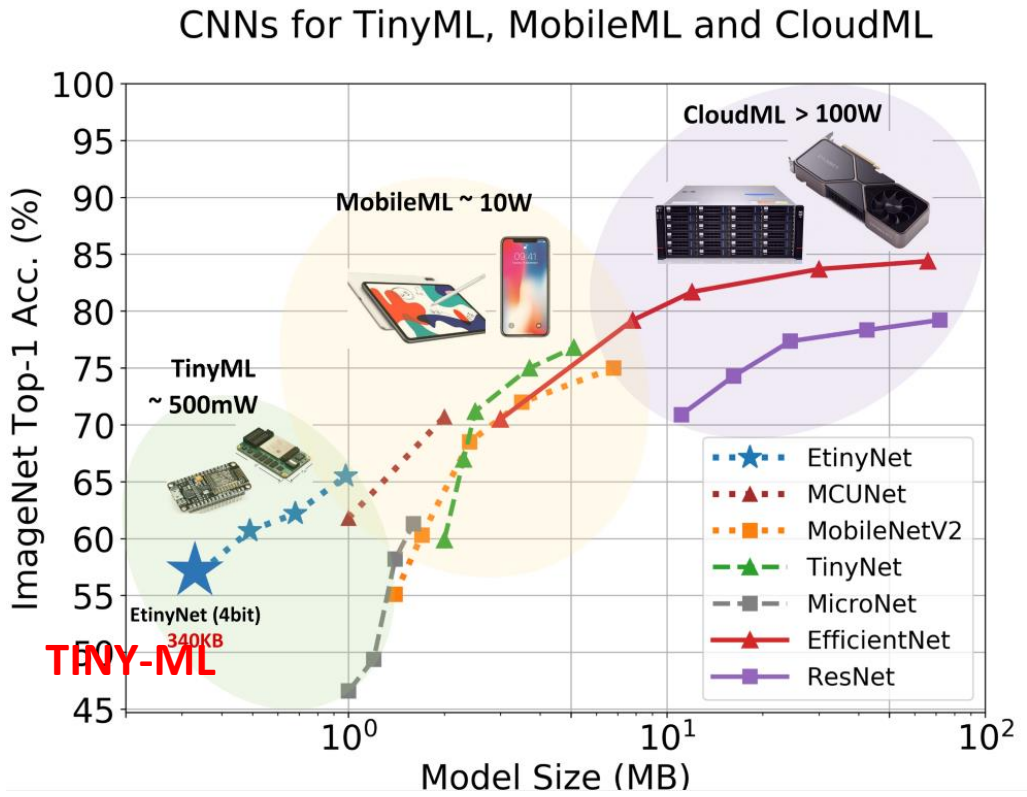
Challenges, motivation, solutions



Signal recognition on TinyML (FE + classifier) should fit into low power, resources constrained MCU

Traditional **FE** = MFCC (e.g. in Edge Impulse) – **200-300 ms** typical – lets very small computation space for classifiers

Proposed FE = iRDT - 9-18ms typical – similar performance



From: EtinyNet: Extremely Tiny Network for TinyML, AAAi 2022

Applicative areas

Medical: EEG, ECG, Phono-cardiograms, Voice-pathology

Personal assistants (keyword spotting, etc.)

Environmental monitoring (chainsaw noises, animals etc.)

Structural health monitoring (presence of cracks in concrete etc)

Emotional speech recognition (detection of anger / stress etc.)

Example from Wagar et al. "**Design of a Speech Anger Recognition System on Arduino Nano 33 BLE Sense", 2021,**



Fig. 6. Implementation of Speech Anger Recognition System

In the last 5-10 years there is an explosion of papers dealing with TinyML signal classification.
Most are still using computationally complex MFCC;
A recent paper in 2026 (Wang et al., "A Sensor-Based TinyML Acoustic Monitoring System for Edge-Side Animal Sound Recognition on Resource-Constrained Microcontrollers") gives the following conclusion:

The measured post-window latency was approximately 303 ms under acoustic playback conditions. This latency was dominated by MFCC feature extraction, whereas TensorFlow Lite Micro invocation required only a small fraction of the total post-window processing time. Therefore, further improvement in real-time responsiveness should focus primarily on the front-end feature-extraction pipeline, including FFT computation, Mel filter-bank mapping, frame-level scheduling, and memory-access patterns. Reducing only the classifier size would have limited impact on the total post-window latency unless the MFCC computation stage is also optimized.

In the next we demonstrate for several case study that proposed iRDT gives comparable overall performance at much lower computational complexity

iRDT Algorithm - energy of changes for *m* channels

The main idea of the processing algorithm (called next iRDT) is depicted in Fig.1
The code for iRDT is written in Python, uses the NUMBA JIT compiler for speed (as well as the LIBROSA MFCC code) and is available online at
https://github.com/radu-dogaru/rdt_transform_for_tiny_ml_signal_classifiers

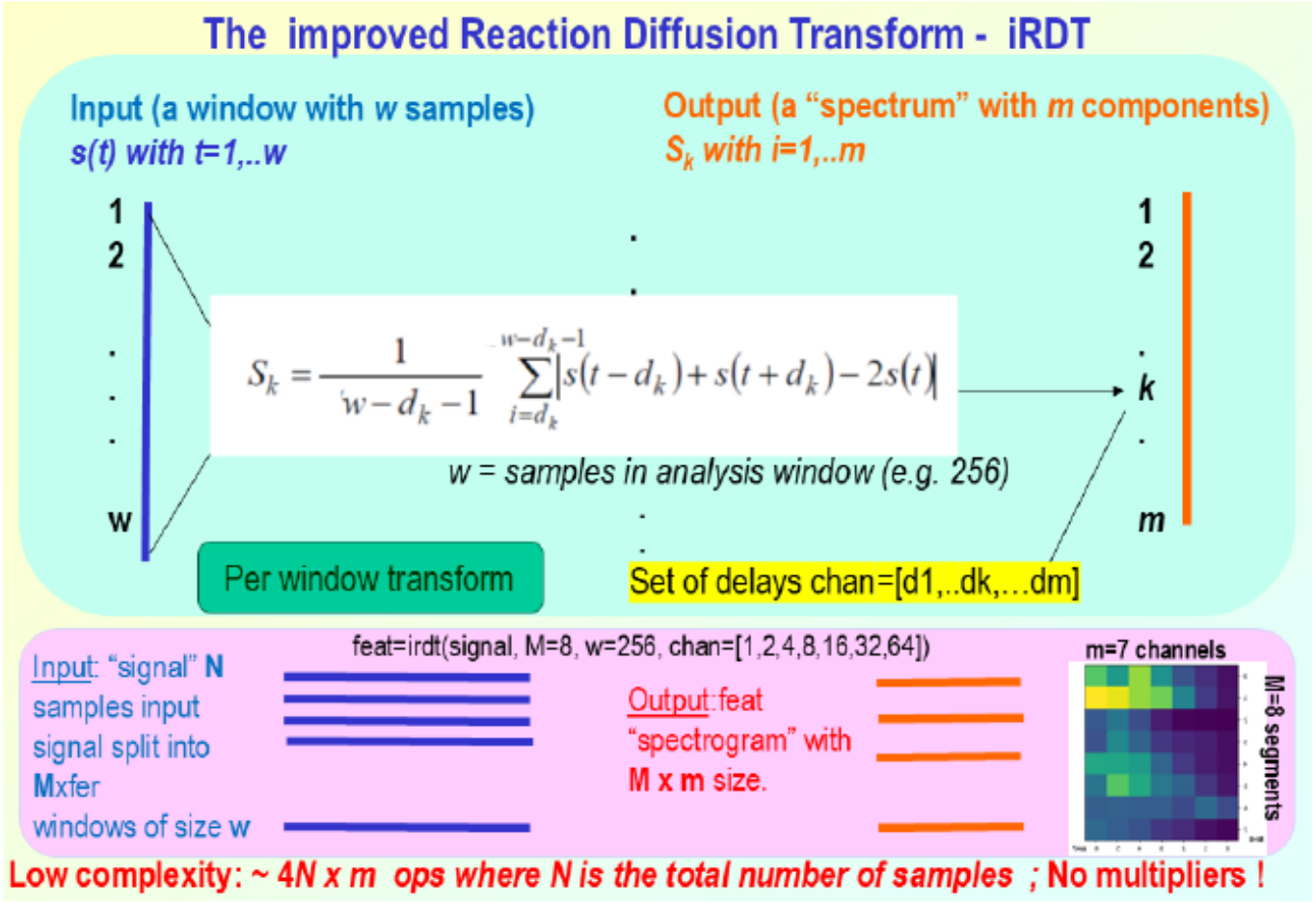


Figure 1: The algorithm defining the improved RDT transform (iRDT)

Algorithm 1 iRDTv (variable number M of segments)

Require: signal, chan, win_per_segm, w
Ensure: M, Feat_spec

1: $m \leftarrow$ length(chan), windows $\leftarrow \lfloor$length(signal)$/w \rfloor$
2: $M \leftarrow \lfloor$windows/win_per_segm$\rfloor$
3: Feat_spec $\leftarrow$ zeros(M, m) ▷ 32-bit int
4: spectrum $\leftarrow$ empty(windows, m) ▷ 32-bit int
5: limit $\leftarrow \lfloor w/4 \rfloor$, base $\leftarrow 0$ ▷ In MCU implementations lines 1-2 (divisions) are precomputed

6: **for** $i = 0$ **to** windows $- 1$ **do**
7: **for** $k = 0$ **to** $m - 1$ **do**
8: delay $\leftarrow$ chan$[k]$, spik $\leftarrow 0$
9: **for** $t =$ limit **to** $w -$ limit $- 1$ **do**
10: spik $\leftarrow$ spik $+$ |signal[base $+ t -$ delay] $+$ signal[base $+ t +$ delay] $-$ 2signal[base $+ t$]|
11: **end for**
12: spectrum$[i, k] \leftarrow$ spik
13: **end for**
14: base $\leftarrow$ base $+ w$
15: **end for**

16: base $\leftarrow 1$ ▷ To avoid padding, ignore first/last spectral lines
17: **for** $k = 0$ **to** $M - 2$ **do**
18: **for** col $= 0$ **to** $m - 1$ **do**
19: F_k_col $\leftarrow 0$
20: **for** $i =$ base **to** base $+$ win_per_segm $- 1$ **do**
21: F_k_col $\leftarrow$ F_k_col $+$ spectrum$[i,$ col$]$
22: **end for**
23: Feat_spec$[k,$ col$] \leftarrow$ F_k_col
24: **end for**
25: base $\leftarrow$ base $+$ win_per_segm
26: **end for**

27: **return** M, Feat_spec

iRDT Algorithm – comparison with MFCC

An effective computation of both iRDT and MFCC-13 (FFT_SIZE= 2048 and HOP_SIZE=512) for the same signal sequence with 29691 samples is presented in Fig. 2.

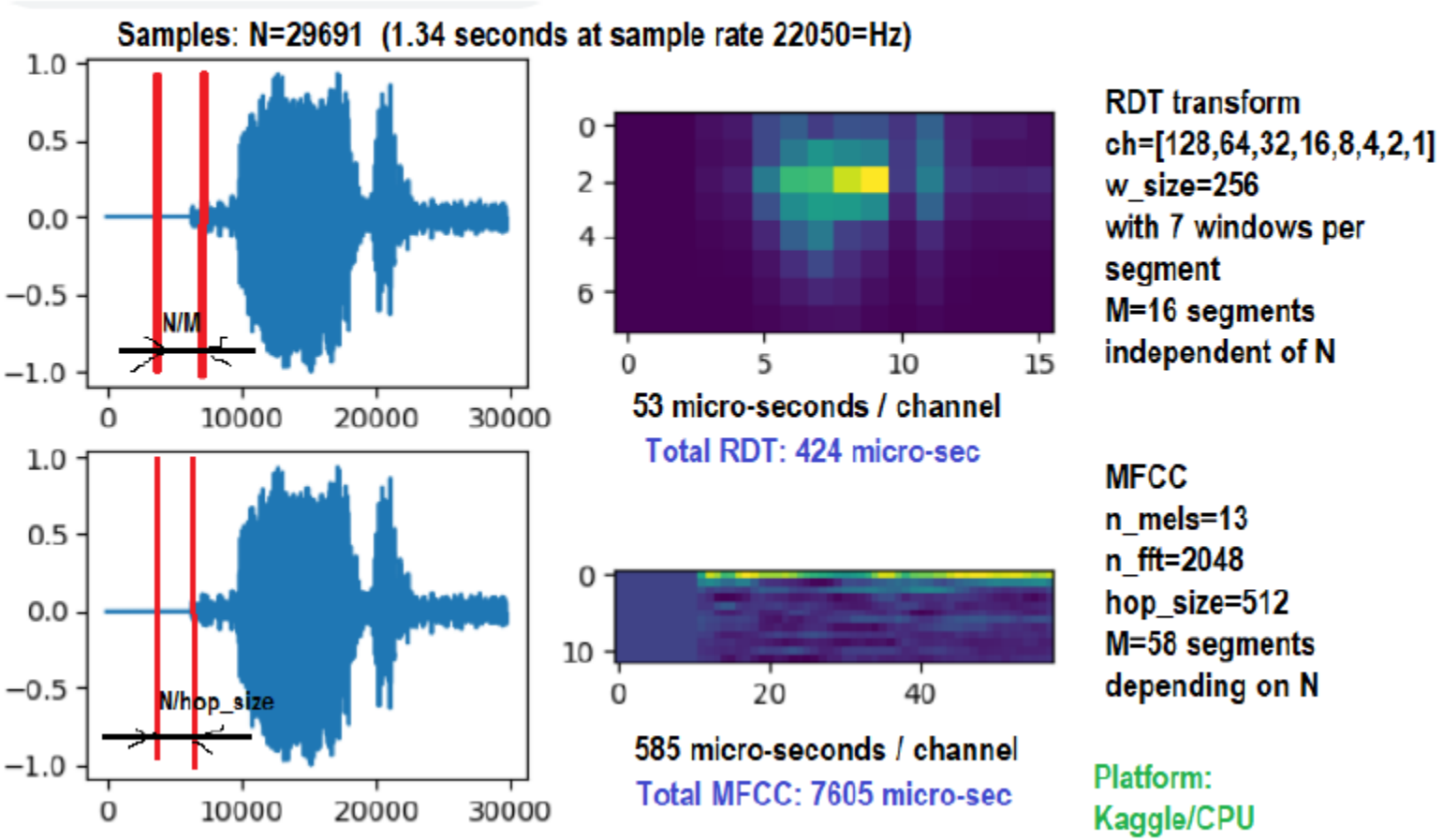


Fig.2: Running both iRDT and MFCC on the same computing platform

TinyML Classifiers

MODEL ADA Model size: 1.547 k MACs: 1.561 k

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 22, 10, 1)	0
flatten (Flatten)	(None, 220)	0
dense (Dense)	(None, 7)	1,547

Very low complexity – may work for some signal classes; Ignore the 2D corelations specific to the (image) spectrogram

MODEL MLP-10 Model size: 2.287 k MACs: 2.296 k

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 22, 10, 1)	0
flatten_1 (Flatten)	(None, 220)	0
dense_1 (Dense)	(None, 10)	2,210
dense_2 (Dense)	(None, 7)	77

MODEL: HVTINY-20; Model size: 9.667 k MACs: 443.901 k

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 22, 10, 1)	0
conv2d (Conv2D)	(None, 22, 10, 20)	320
re_lu (ReLU)	(None, 22, 10, 20)	0
depthwise_conv2d (DepthwiseConv2D)	(None, 22, 10, 20)	320
re_lu_1 (ReLU)	(None, 22, 10, 20)	0
depthwise_conv2d_1 (DepthwiseConv2D)	(None, 22, 10, 20)	320
max_pooling2d (MaxPooling2D)	(None, 11, 5, 20)	0
separable_conv2d (SeparableConv2D)	(None, 11, 5, 80)	1,980
max_pooling2d_1 (MaxPooling2D)	(None, 6, 3, 80)	0
separable_conv2d_1 (SeparableConv2D)	(None, 6, 3, 60)	6,060
max_pooling2d_2 (MaxPooling2D)	(None, 3, 2, 60)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 60)	0
batch_normalization (BatchNormalization)	(None, 60)	240
dropout (Dropout)	(None, 60)	0
dense_3 (Dense)	(None, 7)	427

Convolutional NN but still compact for TinyML solutions – resolves quite well 2D spectrograms (Dogaru et. al, 2025)

Novel model HVTINY-CNN

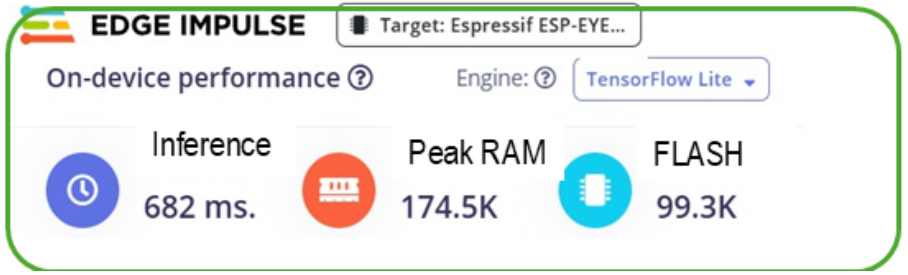
Model HVTINY_N_0_d:0.0_[1, 4, 2, 1]_k:30_c:3
48x48x3 input images

Layer (type)	Output Shape	Param #
input_layer_10 (InputLayer)	(None, 48, 48, 3)	0
conv2d_14 (Conv2D)	(None, 48, 48, 30)	840
re_lu_14 (ReLU)	(None, 48, 48, 30)	0
depthwise_conv2d_7 (DepthwiseConv2D)	(None, 48, 48, 30)	300
re_lu_15 (ReLU)	(None, 48, 48, 30)	0
depthwise_conv2d_8 (DepthwiseConv2D)	(None, 48, 48, 30)	300
max_pooling2d_18 (MaxPooling2D)	(None, 24, 24, 30)	0
separable_conv2d (SeparableConv2D)	(None, 24, 24, 120)	3,990
max_pooling2d_19 (MaxPooling2D)	(None, 12, 12, 120)	0
separable_conv2d_1 (SeparableConv2D)	(None, 12, 12, 60)	8,340
max_pooling2d_20 (MaxPooling2D)	(None, 6, 6, 60)	0
separable_conv2d_2 (SeparableConv2D)	(None, 6, 6, 30)	2,370
max_pooling2d_21 (MaxPooling2D)	(None, 3, 3, 30)	0
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 30)	0
batch_normalization_14 (BatchNormalization)	(None, 30)	120
dropout (Dropout)	(None, 30)	0
dense_5 (Dense)	(None, 2)	62
softmax_3 (Softmax)	(None, 2)	0

1 Nonlinear (2 levels) convolutional macroblock with *k=30* filters

c=3 separable convolutional macroblocks Number of filters according to profile [4*30, 2*30, 1*30]

Classifier



Nonlinear layer inspired from **VRES-CNN** (2024 - https://github.com/radu-dogaru/vres-cnn)
Very good for medium-complexity platforms

VRES-CNN MACRO-LAYER

Layer (type)	Output Shape	Param #
separable_conv2d_9 (SeparableConv2D)	(None, 32, 32, 20)	107
activation_3 (Activation)	(None, 32, 32, 20)	0
separable_conv2d_10 (SeparableConv2D)	(None, 32, 32, 20)	600
add_6 (Add)	(None, 32, 32, 20)	0
batch_normalization_6 (BatchNormalization)	(None, 32, 32, 20)	80
max_pooling2d_6 (MaxPooling2D)	(None, 16, 16, 20)	0
dropout_6 (Dropout)	(None, 16, 16, 20)	0

NL additional layers (here NL = 1)

permanent layer NL=0

But:
- depthwise conv2d are used to reduce complexity
- There is only one “nonlinear” macro-block

HVTINY vas tested so far against VRES-CNN on many other datasets producing MCU-fit solutions with accuracy losses of a few percents.
Available: https://github.com/radu-dogaru/hvtiny
Very good for MCU-oriented platforms

Case study 1: Emotional speech recognition (ADA)

Dataset TESS: A collection of 2,800 audio files consisting of spoken emotional utterances recorded by two actresses (aged 26 and 64) speaking target words embedded in the carrier phrase "Say the word [WORD]". There are 7 categories namely: anger, disgust, fear, happiness, pleasant surprise, sadness and neutral. Best validation accuracy reported so far is 100%.

Table 3: Classification accuracies using various iRDT and MFCC signal processors with simplest ADA classifier

Signal processor	*M*	*w*	*m*	Chan	Acc (%) Valid set	k-MAC ADA Classifier
iRDT-a	20	512	13	[1, 2, 3, 4, 6, 8, 12, 16, 32, 64, 128, 256, 512]	97.5	1.82
iRDT-b	16	512	16	[2, 4, 8, 16, 24, 32, 40, 48, 56, 64, 80, 96, 112, 128, 256, 512]	98.2	1.8
iRDT-c	20	512	16	--"-- (as above)	98.75	2.24
iRDT-d	20	512	22	[1, 2, 4, 8, 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 104, 112, 120, 128, 160, 224, 256]	98.93	3.08
iRDT-e	22	512	22	--"-- (as above)	**99.11**	**3.39**
iRDT-f	43	256	16	[2, 4, 8, 16, 24, 32, 40, 48, 56, 64, 80, 96, 112, 128, 256, 512]	99.11	4.8
iRDT-g	43	256	22	[1, 2, 4, 8, 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 104, 112, 120, 128, 160, 224, 256]	99.11	6.63
iRDT-h	20	512	9	[1,2,4,8,16,32,64,128,256]	97.85	**1.26**
MFCC-13	129	2048	13	---------	85.89	11.7
MFCC-16	129	2048	16	---------	81.6	14.4

Optimized iRDT (iRDT-e) allows a very good accuracy 99.11% with a very simple classifier (overall signal classification pipline has only 1760 K-flops (with no multipliers in the FE)

In contrast **MFCC gives at most 85.89% accuracy**

100% accuracy is obtained with **iRDT-g + HVTINY-20** (the same classifier with MFCC gives only 45%) **with overall 6492 Kflops**
100% with MFCC requires a more complex CNN (not supported in MCU platforms over 10M-flops)

Case study 2: Isolated audio events detection

Dataset from Signal Processing Group at UT Cluj – 46 sound events for TIAGo service robots to assist people in a household environment – Baseline: In [27] the authors did a careful optimization of MFCC feature extractors reporting the best validation accuracy of **99.27% when using a MFCC with 20 coefficients combined with a k-NN classifier**.

Table 4: Signal classifiers using iRDT for the TIAGO1 dataset

iRDT model	*M*	*w*	*m*	Chan	Classifier	Classifier k-MAC	Validation accuracy
iRDT-X 48 Khz	16	128	18	[2, 4, 8, 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 104, 112, 120, 128]	**ADA**	**13.3**	**91.08**
--"--	-"-	--"--	-"-	--"--	MLP-100	33.6	94.94
--"--		--"--	-"-	--"--	HVTINY-20	558k	96.86
iRDT-A 48 kHz	16	128	8	[1, 2, 4, 8, 16, 32, 64, 128]	**ADA**	**6**	**90.12**
--"--	-"-	--"--	-"-	--"--	MLP-100	17.59	95.66
--"--	-"-	--"--	-"-	--"--	HVTINY-20	243.7	**99.52**
iRDT-A 24 kHz	16	128	8	[1, 2, 4, 8, 16, 32, 64, 128]	ADA	6	88.96
--"--	-"-	--"--	-"-	--"--	MLP-100	17.59	95.66
--"--	-"-	--"--	-"-	--"--	HVTINY-20	243.7	98.55
iRDT-A 16 kHz	16	128	8	[1, 2, 4, 8, 16, 32, 64, 128]	ADA	6	85.78
--"--	-"-	--"--	-"-	--"--	MLP-100	17.59	94.21
--"--	-"-	--"--	-"-	--"--	**HVTINY-20**	**243.7**	**99.52**

Optimized iRDT (iRDT-X) at 48Khz sampling rate allows quite good accuracy 91.08% with a very simple classifier

Still very simple classifier (MLP) with lower sampling rate (24Khz) ensures 95.66% accuracy with iRDT-A

Optimized iRDT (iRDT-A) at lower 16khz sampling rate (reducing overall complexity) + CNN (HVTINY-20) ensures 99.52% accuracy ! + only 1512 Kflops

[27] Telembici T., Grama L., Muscar L., Rusu C., *Results on the MFCC extraction for improving audio capabilities of TIAGo service robot*, Proceedings of the 2021 International Conference on Speech Technology and Human-Computer Dialogue (SpeD), Bucharest, Romania, 2021, p. 57-61.

Case study 3: Structural health monitoring

In [28] the authors claim to provide the first portable recognition device for AE structure health monitoring using a compact TinyML-platform (Arduino Nano BLE33 Sense) capable of recognizing **3 classes of possible damage; namely, 0-tensile, 1-shear, and 2-mixed.** Acoustic signals are **time-series of 1000 samples for each class, received from the ultrasonic transducer**. The data set used to evaluate their solution is publicly available from [29].
Their solution achieved 99.7% accuracy using a CNN network directly to input signals. Their CNN model still requires some 20k-paramaters and after deploying the model using INT8 LiteRT quantization the accuracy reduced to 99.6%. **Their model has a computational complexity of 2300 Kflops**

Our preliminary results on using iRDT on the same dataset indicate the following best solution:
iRDT configuration: w=64, chan= [1, 2, 3, 4, 6, 8, 12, 16, 20, 24, 32, 40, 48, 64] i.e. m=14, M=8;
the iRDT processor complexity is 2x1000x14=28 Kflops.
Classifier configuration: MLP-20 with Model size: 2.323 kpars and 2.3 kMAC (or 4.6 Kflops)

With this setup, **a better validation accuracy of 99.87% is obtained**, while the entire system has now only **32.6 Kflops (77 times less than in the proposed CNN)** and requires **memory storage only for 2.3Kpars (9 times less than in the CNN-based solution).** In addition, **there is no convolution operator needed anymore!**

[28] Zhang, Yuxuan, et al., Leveraging acoustic emission and machine learning for concrete materials damage classification on embedded devices, IEEE Transactions on Instrumentation and Measurement, 2023, 72: pp. 1-8.
[29] Siracusano, Giulio, et al., Automatic crack classification by exploiting statistical event descriptors for deep learning, Applied Sciences 11.24, 2021, 12059.

Case study 4: In-Car Automatic Speech Recognition

Dataset from M. Bucurica [30]: 9 vocal commands inside a motor vehicle (each command has 20 associated signal sequences spoken by the same user, on background noise from the vehicle); Baseline 88% with an older 1D-version of RDT + SVC (shallow vector support classifier).

Table 5: Signal classifiers using iRDT for the auto-voice dataset; In all experiments *w=128, m=8, chan =[1,2,4,8,16,32,64,128]*

iRDT model	*M*	Classifier	Classifier complexity kMAC	Validation accuracy
iRDT-A	8	ADA	**0.58**	**75.86**
iRDT-A	10	ADA	0.72	79.31
iRDT-B	12	ADA	0.87	**81.03**
iRDT-C	14	ADA	1.01	**81.03**
iRDT-D	16	ADA	1.18	79.31
iRDT-E	24	ADA		70.69
iRDT-F	32	ADA	2.31	74.13
iRDT-B	12	MLP-50	5.3	84.48
iRDT-B 44.1 Khz	12	**HVTINY-20**	**181.5**	**100**
		Other sampling rates		
iRDT-B 22.05 kHz	12	**HVTINY-20**	**181.5**	**100**
iRDT-D 16 Khz	16	HVTINY-20	243.7	96.55
iRDT-E 16 Khz	24	**HVTINY-20**	**361.7**	**100**

For this KWS (Keyword Spotting) problem simple classifiers are not enough (accuracies < 85%)

Optimized iRDT (iRDT-E) at 16khz sampling rate (reducing overall complexity) + CNN (HVTINY-20) ensures 100% accuracy + only 1412 Kflops

More recent results on the standard Google KWS dataset indicates competitive results (94.7% on the 12 class version) when employing iRDT

Paper with code and demo at
https://arxiv.org/abs/2608.17108

[30] Bucurica M., Dogaru I., Dogaru R., *Novel Applications of Complexity Inspired RDT Transform for Low Complexity Embedded Speech Recognition in Automotive Environments*, Proceedings of the 2017 21st International Conference on Control Systems and Computer Science (CSCS), Bucharest, Romania, 2017, p. 375-378.

Concluding remarks

iRDT competes favorably with MFCC for usual tasks in signal classification

The advantages of employing iRDT in signal classification are:

a) An easy to understand and platform-wide implementable algorithm using very simple arithmetic operators;

b) One order of magnitude times faster than MFCC, removes energy inefficient operators such as multipliers and cosine

c) Enables the use of simple, non-convolutional classifiers such as Adaline (ADA) or Multi-Layer Perceptrons (MLP) with a good compromise between complexity and performance;

d) Generated feature sizes are usually much smaller than in MFCC, thus reducing the complexity of lightweight CNN classifiers required when Adaline or MLP is not sufficient for providing acceptable accuracy.



Thank you !

Do you have any questions?

radu.dogaru@upb.ro